DMBI – CONTINUOUS ASSESSMENT 1

CASE STUDY-BASED DATA MINING ANALYSIS

From Data to Decisions:
Predicting Viewer Drop-Off in OTT Streaming Platforms Using Data Mining

Subject: Data Mining and Business Intelligence

Submitted To: _____

Submitted By: Darshan Uttam Karamchandani

Class / Division: D15A

Roll Number: 18

Academic Year: 2026–2027

1. Introduction, Problem Statement and Business Objective:

Over-the-top (OTT) streaming platforms provide films, series and episodes through internet-connected devices. A platform may lose viewer attention when users stop watching an episode or do not continue viewing. This case study examines viewer drop-off using historical OTT viewing-behaviour records.

1.1 Problem Statement

The problem is to identify viewing and content characteristics associated with a viewer dropping off. The target variable drop_off has two classes: retained (0) and dropped off (1).

1.2 Business Objective

The objectives are to:

identify patterns associated with viewer drop-off;

train and compare two classification models;

evaluate models using accuracy, precision, recall, F1-score and confusion matrices;

provide evidence that could guide content review, viewer-engagement improvements and future retention experiments.

The model is intended to support analysis, not to automatically make decisions about individual viewers.

2. Data Understanding and Data Source:

The analysis uses the CSV file "JioHotstar_Preprocessed_Dataset .csv", loaded into Google Colab using Python. The inspected dataset contains 33,171 rows and 24 columns.

Source limitation: the file name refers to JioHotstar, but platform values include services such as Netflix and Hulu. Therefore, the dataset is described here as an OTT viewing-behaviour dataset, not as verified data collected exclusively from JioHotstar. The original source and collection method were not independently verified.

2.1 Important Attributes

Attribute	Description / Role
platform	Streaming platform represented in a record.
genre	Content genre.
release_year	Content release year.
season_number / episode_number	Season and episode identifiers where applicable.
episode_duration_min	Episode duration in minutes.
pacing_score / hook_strength	Content indicators describing pacing and opening hook.
dialogue_density / visual_intensity	Content characteristics.

avg_watch_percentage	Average percentage of content watched.
pause_count / rewind_count / skip_intro	Viewing-interaction behaviour.
cognitive_load / attention_required / night_watch_safe	Additional content/viewing descriptors.
drop_off	Target: 0 = retained; 1 = dropped o

2.2 Target Class Distribution

Target class	Count	Percentage
Retained (0)	28,367	85.52%
Dropped off (1)	4,804	14.48%
Total	33,171	100%

Retained records form the majority class. Therefore, accuracy alone is not enough to assess performance; precision, recall and F1-score for the dropped-off class are also important.

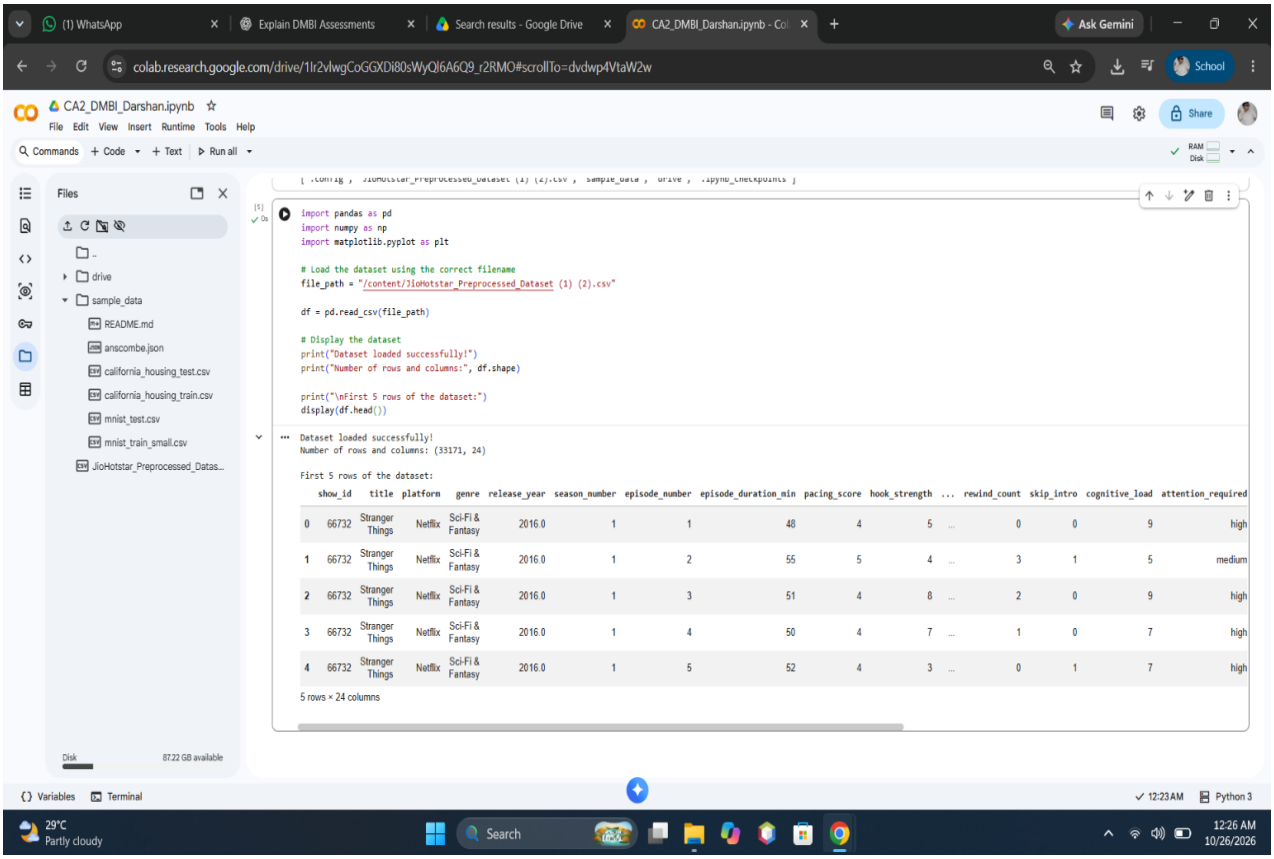


Figure 1: Preview of the OTT Viewing-Behaviour Dataset

Inference: The CSV file was loaded successfully in Google Colab. The preview shows sample records and attributes available for the viewer drop-off analysis.

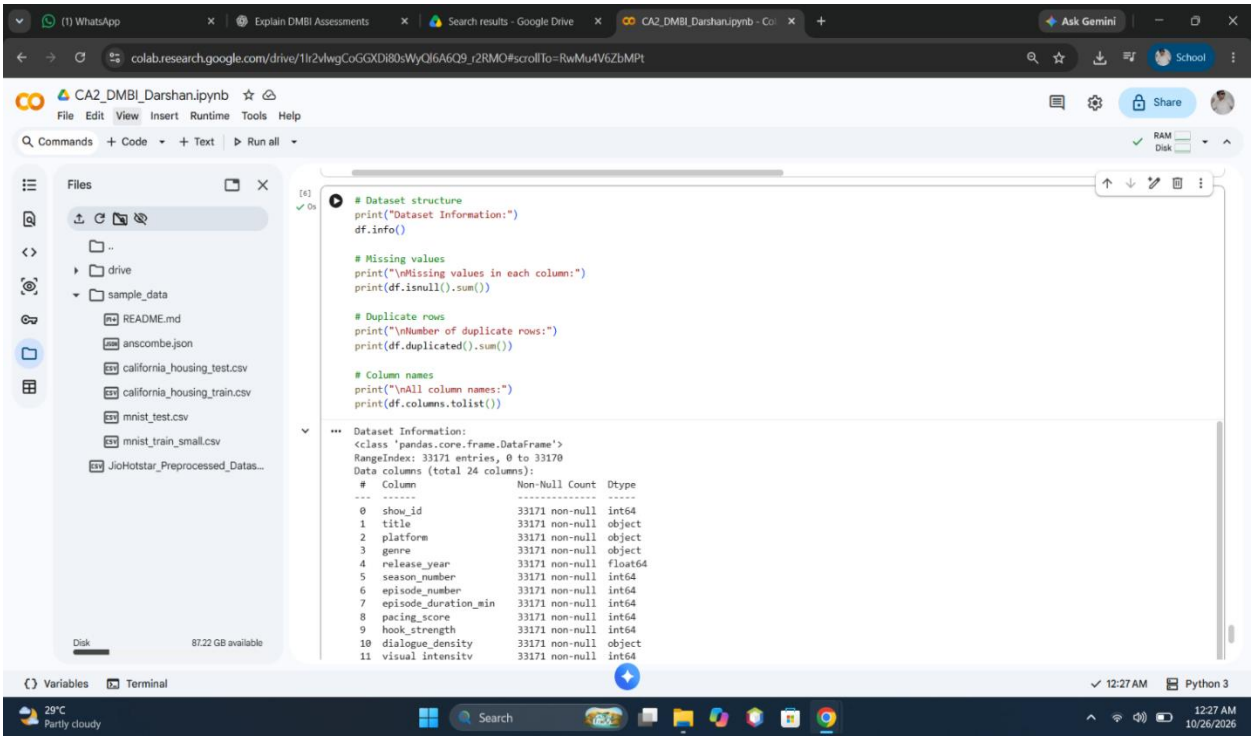


Figure 2: Dataset Structure and Information

```
print(df.duplicated().sum())

# Column names
print("\nAll column names:")
print(df.columns.tolist())
```

```
 2   platform                33171 non-null  object
 3   genre                   33171 non-null  object
 4   release_year            33171 non-null  float64
 5   season_number           33171 non-null  int64
 6   episode_number          33171 non-null  int64
 7   episode_duration_min    33171 non-null  int64
 8   pacing_score            33171 non-null  int64
 9   hook_strength           33171 non-null  int64
 10  dialogue_density        33171 non-null  object
 11  visual_intensity        33171 non-null  int64
 12  avg_watch_percentage    33171 non-null  int64
 13  pause_count             33171 non-null  int64
 14  rewind_count            33171 non-null  int64
 15  skip_intro              33171 non-null  int64
 16  cognitive_load          33171 non-null  int64
 17  attention_required      33171 non-null  object
 18  night_watch_safe        33171 non-null  int64
 19  drop_off                33171 non-null  int64
 20  drop_off_probability    33171 non-null  float64
 21  retention_risk          33171 non-null  object
 22  dataset_version         33171 non-null  object
 23  watch_engagement_level  33171 non-null  object
dtypes: float64(2), int64(14), object(8)
memory usage: 6.1+ MB

Missing values in each column:
show_id                 0
title                   0
platform                0
genre                   0
```

```
Missing values in each column:
show_id                  0
title                    0
platform                 0
genre                    0
release_year             0
season_number            0
episode_number           0
episode_duration_min     0
pacing_score             0
hook_strength            0
dialogue_density         0
visual_intensity         0
avg_watch_percentage     0
pause_count              0
rewind_count             0
skip_intro               0
cognitive_load           0
attention_required       0
night_watch_safe         0
drop_off                 0
drop_off_probability     0
retention_risk           0
dataset_version          0
watch_engagement_level   0
dtype: int64

Number of duplicate rows:
0

All column names:
['show_id', 'title', 'platform', 'genre', 'release_year', 'season_number', 'episode_number', 'episode_duration_min', 'pacing_score', 'hook_strength', 'di
```

[7]
✓ 0s

```python
# Check missing values clearly
missing_values = df.isnull().sum()

print("Total missing values:", missing_values.sum())
print("\nColumns containing missing values:")
display(missing_values[missing_values > 0])

print("\nDataset dimensions:", df.shape)
print("Duplicate rows:", df.duplicated().sum())

# Check the target column
print("\nUnique values in drop_off:")
print(df["drop_off"].value_counts(dropna=False))
```

```
Total missing values: 0

Columns containing missing values:
```

	0

dtype: int64

```
Dataset dimensions: (33171, 24)
Duplicate rows: 0

Unique values in drop_off:
drop_off
0    28367
1     4804
Name: count, dtype: int64
```

Inference: The dataset contains 33,171 records and 24 columns. This information helps establish the size and structure of the data available for analysis and identifies the attributes that may be useful for building classification models.

```python
import matplotlib.pyplot as plt

# Count each target class
dropoff_counts = df["drop_off"].value_counts().sort_index()

print("Drop-off distribution:")
print(dropoff_counts)

# Create a bar chart
plt.figure(figsize=(7, 5))
dropoff_counts.plot(kind="bar")

plt.title("Distribution of Viewer Drop-Off")
plt.xlabel("Drop-Off Class (0 = Retained, 1 = Dropped Off)")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
Drop-off distribution:
drop_off
0    28367
1     4804
Name: count, dtype: int64
```

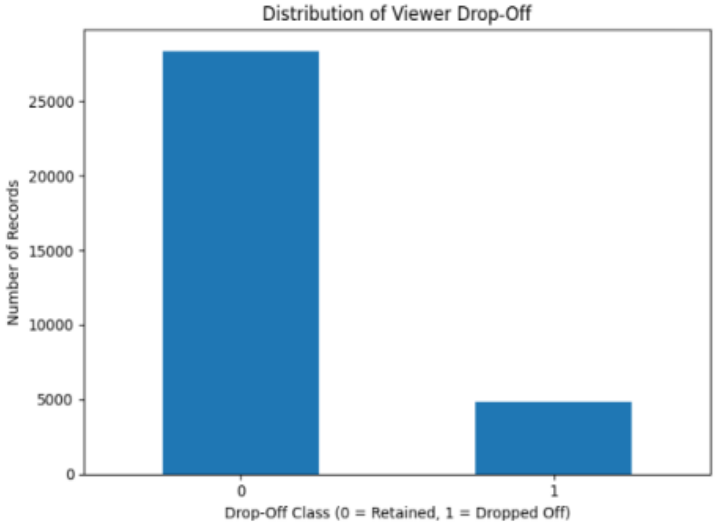


Figure 3: Distribution of Retained and Dropped-Off Viewers

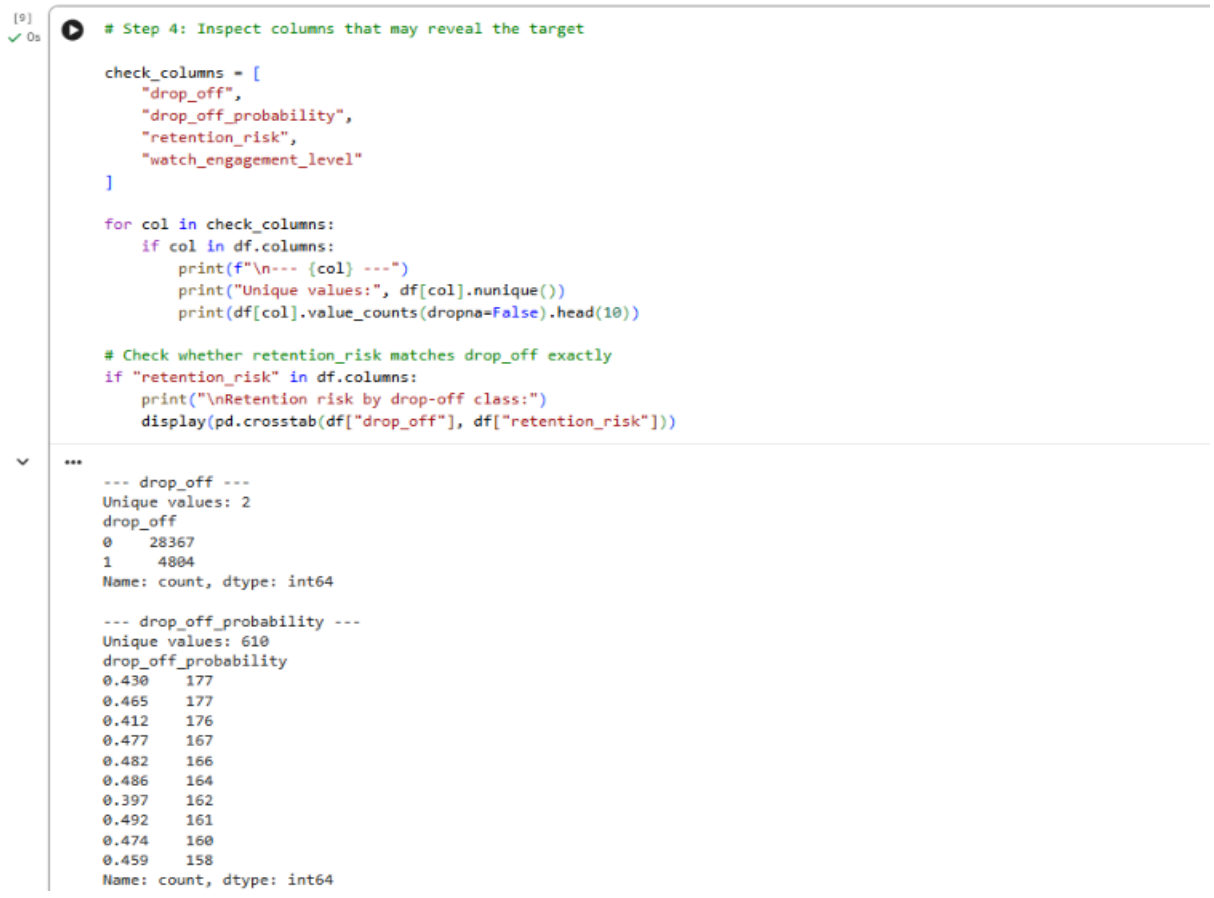
Inference: The target variable contains 28,367 retained records (85.52%) and 4,804 dropped-off records (14.48%). This indicates that the dataset has an imbalanced class distribution, with retained records forming the majority. Therefore, precision, recall and F1-score should also be considered when evaluating the models instead of relying only on accuracy.

3. Data Preprocessing:

The following preprocessing steps were performed:

a) Loaded the CSV into a pandas DataFrame and inspected dimensions, column names and data types.
b) Checked missing values and exact duplicate rows; the inspected output showed none.
c) Separated the target variable drop_off from the input features.
d) Excluded identifier/descriptive columns such as show_id, title and dataset_version.
e) Excluded drop_off_probability, retention_risk and watch_engagement_level because they may reveal or closely encode target information.
f) Identified categorical attributes (including platform, genre, dialogue_density and attention_required) and applied one-hot encoding; numeric features were passed through.
g) Used an 80:20 stratified train-test split with random_state=42.

Potential leakage caution: the timing and meaning of attributes such as avg_watch_percentage must be checked. If a feature is only available after a viewer has already dropped off, it would not be suitable for real-time prediction. Results should be treated as preliminary until feature timing and target leakage are verified.



```
[9]
✓ 0s
# Step 4: Inspect columns that may reveal the target

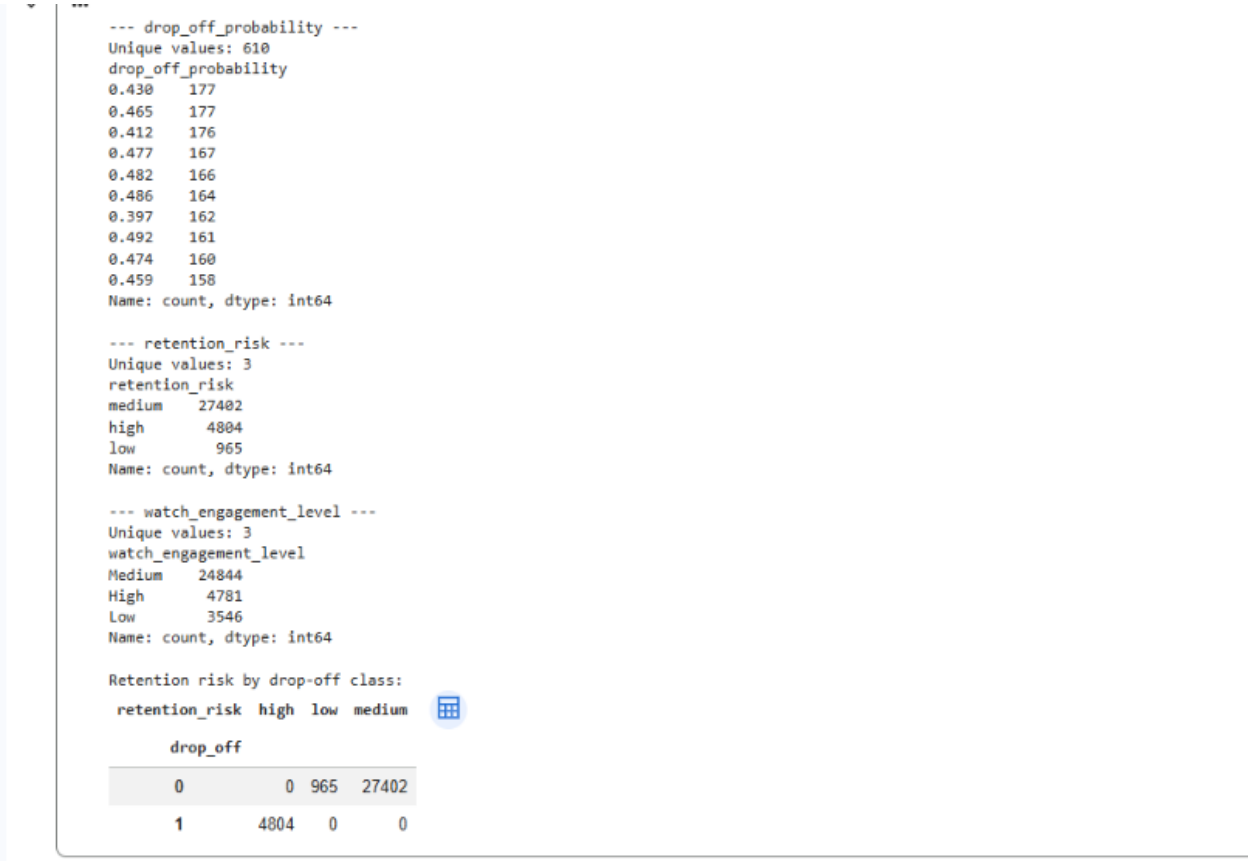
check_columns = [
    "drop_off",
    "drop_off_probability",
    "retention_risk",
    "watch_engagement_level"
]

for col in check_columns:
    if col in df.columns:
        print(f"\n--- {col} ---")
        print("Unique values:", df[col].nunique())
        print(df[col].value_counts(dropna=False).head(10))

# Check whether retention_risk matches drop_off exactly
if "retention_risk" in df.columns:
    print("\nRetention risk by drop-off class:")
    display(pd.crosstab(df["drop_off"], df["retention_risk"]))
```

```
--- drop_off ---
Unique values: 2
drop_off
0    28367
1     4804
Name: count, dtype: int64

--- drop_off_probability ---
Unique values: 610
drop_off_probability
0.430    177
0.465    177
0.412    176
0.477    167
0.482    166
0.486    164
0.397    162
0.492    161
0.474    160
0.459    158
Name: count, dtype: int64
```



```
--- drop_off_probability ---
Unique values: 610
drop_off_probability
0.430    177
0.465    177
0.412    176
0.477    167
0.482    166
0.486    164
0.397    162
0.492    161
0.474    160
0.459    158
Name: count, dtype: int64

--- retention_risk ---
Unique values: 3
retention_risk
medium    27402
high       4804
low         965
Name: count, dtype: int64

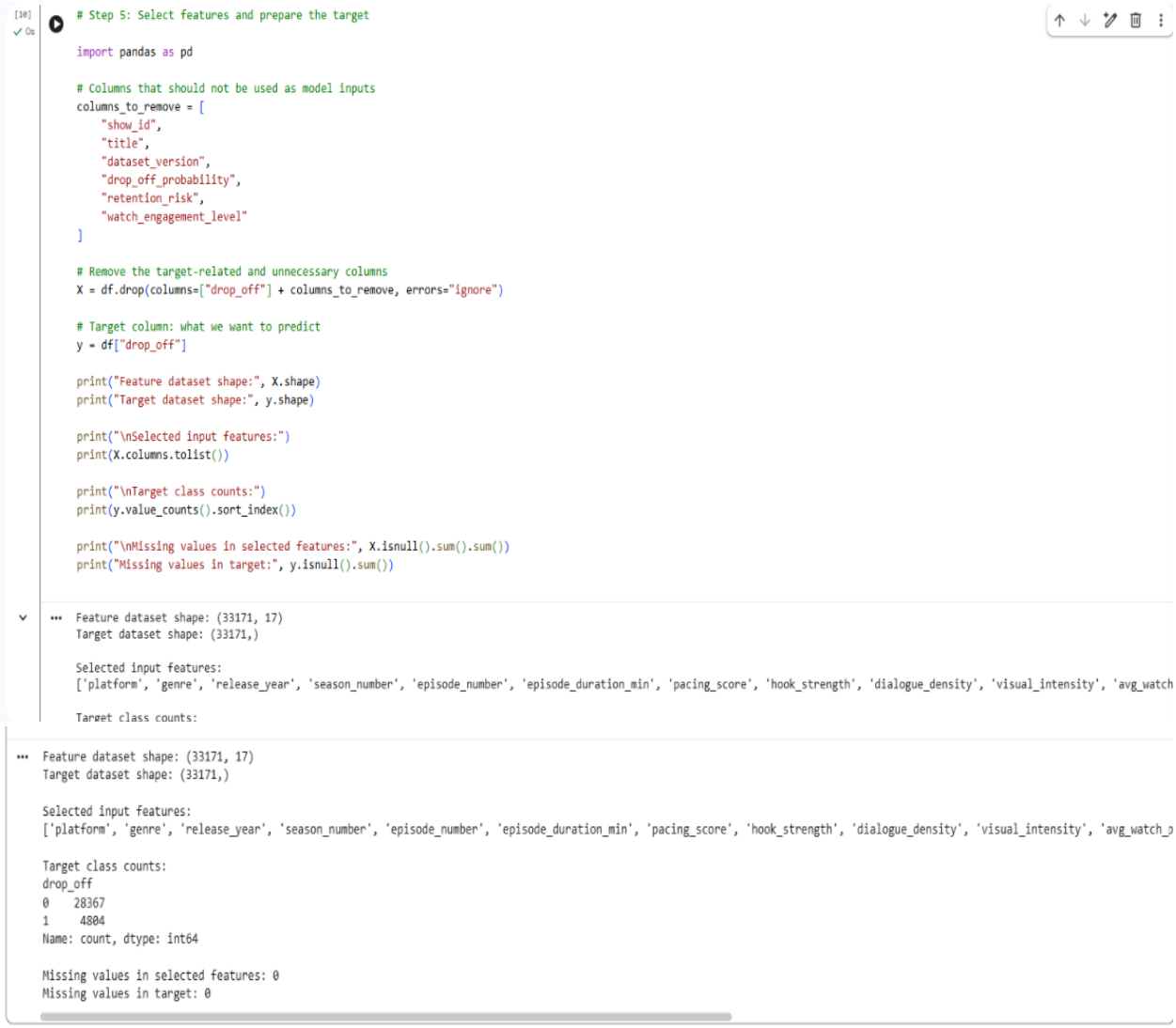
--- watch_engagement_level ---
Unique values: 3
watch_engagement_level
Medium    24844
High       4781
Low        3546
Name: count, dtype: int64

Retention risk by drop-off class:
```

retention_risk	high	low	medium
drop_off			
0	0	965	27402
1	4804	0	0

Figure 4: Target Leakage Check

Inference: This step examines whether any columns contain information that could reveal the target variable or make model evaluation misleading. Columns such as drop_off_probability, retention_risk and watch_engagement_level were excluded from the selected model features. The timing and relationship of the remaining viewing-behaviour attributes should also be checked to ensure that the model does not use information that would only become available after the prediction point.



```
[10]
✓ 0s
# Step 5: Select features and prepare the target

import pandas as pd

# Columns that should not be used as model inputs
columns_to_remove = [
    "show_id",
    "title",
    "dataset_version",
    "drop_off_probability",
    "retention_risk",
    "watch_engagement_level"
]

# Remove the target-related and unnecessary columns
X = df.drop(columns=["drop_off"] + columns_to_remove, errors="ignore")

# Target column: what we want to predict
y = df["drop_off"]

print("Feature dataset shape:", X.shape)
print("Target dataset shape:", y.shape)

print("\nSelected input features:")
print(X.columns.tolist())

print("\nTarget class counts:")
print(y.value_counts().sort_index())

print("\nMissing values in selected features:", X.isnull().sum().sum())
print("Missing values in target:", y.isnull().sum())
```

```
Feature dataset shape: (33171, 17)
Target dataset shape: (33171,)

Selected input features:
['platform', 'genre', 'release_year', 'season_number', 'episode_number', 'episode_duration_min', 'pacing_score', 'hook_strength', 'dialogue_density', 'visual_intensity', 'avg_watch

Target class counts:
```

```
Feature dataset shape: (33171, 17)
Target dataset shape: (33171,)

Selected input features:
['platform', 'genre', 'release_year', 'season_number', 'episode_number', 'episode_duration_min', 'pacing_score', 'hook_strength', 'dialogue_density', 'visual_intensity', 'avg_watch_p

Target class counts:
drop_off
0    28367
1     4804
Name: count, dtype: int64

Missing values in selected features: 0
Missing values in target: 0
```

Figure 5: Data Preprocessing and Feature Selection

Inference: The dataset was prepared for classification by separating the target variable drop_off from the input features. Irrelevant identifier columns and potentially leakage-related columns were excluded. The selected features include platform, genre and viewing-behaviour attributes. Categorical variables were encoded so that the classification algorithms could process them.

```python
# Step 6: Split data into training and testing sets

from sklearn.model_selection import train_test_split

# Split the dataset while keeping the class proportions similar
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)

print("\nTraining target distribution:")
print(y_train.value_counts().sort_index())

print("\nTesting target distribution:")
print(y_test.value_counts().sort_index())
```

```
Training features: (26536, 17)
Testing features: (6635, 17)

Training target distribution:
drop_off
0    22693
1     3843
Name: count, dtype: int64

Testing target distribution:
drop_off
0    5674
1     961
Name: count, dtype: int64
```

Figure 6: Training and Testing Data Split

Inference: The dataset was divided using an 80:20 stratified split. The training set contains 26,536 records, while the testing set contains 6,635 records. Stratification preserves approximately the same target-class proportions in both sets, allowing the models to be evaluated on data not used during training.

4. Identification of the Appropriate Data Mining Task

4.1 Classification

Classification is appropriate because drop_off has two predefined classes: retained (0) and dropped off (1). The models learn from labelled examples and predict a class for unseen records. This is supervised learning. Predictions can identify associations but do not prove that a content feature causes drop-off.

5. Selected Algorithms and Justification

5.1 Decision Tree Classifier

A Decision Tree makes predictions through feature-based decision rules. It is useful here because its rules can be inspected and explained. A maximum tree depth was used to help control model complexity.

5.2 Naïve Bayes Classifier

Naïve Bayes is a probabilistic algorithm based on Bayes' theorem. It is fast and provides a useful baseline for comparison. Gaussian Naïve Bayes was used after categorical attributes were encoded.

5.3 Comparison Rationale

Criterion	Decision Tree	Naïve Bayes
Approach	Learns decision rules and feature splits.	Uses probabilities and Bayes' theorem.
Strength	Can represent complex patterns and is interpretable.	Fast and useful as a baseline.
Limitation	May overfit if not controlled.	Assumes conditional independence of features.
Role	Main candidate for further interpretation.	Baseline for comparison.

```
12]
✓ 1s
```

```python
# Step 7: Preprocess features and train both models

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Identify categorical and numerical columns
categorical_columns = X_train.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

numerical_columns = [
    col for col in X_train.columns
    if col not in categorical_columns
]

print("Categorical features:", categorical_columns)
print("Numerical features:", numerical_columns)

# Convert categories to numbers; keep numerical features as they are
preprocessor = ColumnTransformer(
    transformers=[
        ("categorical",
         OneHotEncoder(handle_unknown="ignore", sparse_output=False),
         categorical_columns),
        ("numerical", "passthrough", numerical_columns)
    ],
    remainder="drop"
)

# Model 1: Decision Tree
decision_tree = Pipeline([
    ("preprocessing", preprocessor),
    ("model", DecisionTreeClassifier(
```

```python
        ( categorical ,
         OneHotEncoder(handle_unknown="ignore", sparse_output=False),
         categorical_columns),
        ("numerical", "passthrough", numerical_columns)
    ],
    remainder="drop"
)

# Model 1: Decision Tree
decision_tree = Pipeline([
    ("preprocessing", preprocessor),
    ("model", DecisionTreeClassifier(
        max_depth=5,
        random_state=42,
        class_weight="balanced"
    ))
])

# Model 2: Gaussian Naive Bayes
naive_bayes = Pipeline([
    ("preprocessing", preprocessor),
    ("model", GaussianNB())
])

# Train both models using training data only
decision_tree.fit(X_train, y_train)
naive_bayes.fit(X_train, y_train)

print("\nBoth models trained successfully!")
print("Decision Tree training complete.")
print("Naive Bayes training complete.")
```

```
Categorical features: ['platform', 'genre', 'dialogue_density', 'attention_required']
Numerical features: ['release_year', 'season_number', 'episode_number', 'episode_duration_min', 'pacing_score', 'hook_strength', 'visual_intensity', 'avg_watch_percentage', 'pause_co

Both models trained successfully!
Decision Tree training complete.
Naive Bayes training complete.
```

Figure 7: Training the Decision Tree and Naïve Bayes Models

Inference: Both Decision Tree and Naïve Bayes classifiers were trained using the prepared training dataset. Using two algorithms makes it possible to compare their predictive performance on the same testing dataset and determine which is more suitable for identifying viewer drop-off.

6. Data Mining Process

Problem → Data → Preprocessing → Algorithm → Results → Evaluation → Insights → Decision

Stage	Application in this case study
Problem	Understand and analyse viewer drop-off in OTT viewing.
Data	Load and inspect the 33,171-record CSV dataset.
Preprocessing	Inspect data quality, select features, encode categorical columns and split data.
Algorithm	Train Decision Tree and Naïve Bayes classifiers.
Results	Generate predictions for the test records.
Evaluation	Compare accuracy, precision, recall, F1-score and confusion matrices.
Insights	Determine which model better balances drop-off detection and false warnings.
Decision	Use findings to guide further investigation and retention experiments.

7. Available Results and Evaluation Measures

Both models were evaluated on the same test set of 6,635 records. Precision, recall and F1-score in the main comparison refer to the dropped-off class (class 1).

7.1 Model Comparison

Evaluation measure	Decision Tree	Naïve Bayes
Accuracy	95.80%	72.61%
Precision (dropped-off class)	78.70%	34.26%
Recall (dropped-off class)	97.29%	96.98%
F1-score (dropped-off class)	87.02%	50.64%

The Decision Tree achieved higher accuracy, precision and F1-score. Both models achieved high recall, but Naïve Bayes produced many more false drop-off predictions. These results favour the Decision Tree for further validation, subject to the limitations described above.

7.2 Evaluation Measures

Measure	Meaning in this case
Accuracy	Proportion of all test records classified correctly.
Precision	Of records predicted as dropped off, the proportion actually dropped off.
Recall	Of actual dropped-off records, the proportion correctly identified.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Displays correct and incorrect predictions for each class.

7.3 Confusion Matrices

Rows represent actual classes and columns represent predicted classes.

Decision Tree	Predicted Retained (0)	Predicted Dropped Off (1)
Actual Retained (0)	5,421	253
Actual Dropped Off (1)	26	935
Naïve Bayes	**Predicted Retained (0)**	**Predicted Dropped Off (1)**
Actual Retained (0)	3,886	1,788
Actual Dropped Off (1)	29	932

```python
# Step 8: Evaluate Decision Tree and Naive Bayes

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Make predictions on unseen testing data
dt_predictions = decision_tree.predict(X_test)
nb_predictions = naive_bayes.predict(X_test)

# Evaluate Decision Tree
print("========== DECISION TREE ==========")
print("Accuracy:", round(accuracy_score(y_test, dt_predictions), 4))
print("\nClassification Report:")
print(classification_report(
    y_test, dt_predictions,
    labels=[0, 1],
    target_names=["Retained (0)", "Dropped Off (1)"],
    zero_division=0
))
print("Confusion Matrix:")
print(confusion_matrix(y_test, dt_predictions, labels=[0, 1]))

# Evaluate Naive Bayes
print("\n========== NAIVE BAYES ==========")
print("Accuracy:", round(accuracy_score(y_test, nb_predictions), 4))
print("\nClassification Report:")
print(classification_report(
    y_test, nb_predictions,
    labels=[0, 1],
    target_names=["Retained (0)", "Dropped Off (1)"],
    zero_division=0
))
print("Confusion Matrix:")
print(confusion_matrix(y_test, nb_predictions, labels=[0, 1]))
```


```python
    zero_division=0
))
print("Confusion Matrix:")
print(confusion_matrix(y_test, nb_predictions, labels=[0, 1]))
```


```
========== DECISION TREE ==========
Accuracy: 0.958

Classification Report:
                  precision    recall  f1-score   support

   Retained (0)       1.00      0.96      0.97      5674
Dropped Off (1)       0.79      0.97      0.87       961

       accuracy                           0.96      6635
      macro avg       0.89      0.96      0.92      6635
   weighted avg       0.97      0.96      0.96      6635

Confusion Matrix:
[[5421  253]
 [  26  935]]

========== NAIVE BAYES ==========
Accuracy: 0.7261

Classification Report:
                  precision    recall  f1-score   support

   Retained (0)       0.99      0.68      0.81      5674
Dropped Off (1)       0.34      0.97      0.51       961

       accuracy                           0.73      6635
      macro avg       0.67      0.83      0.66      6635
   weighted avg       0.90      0.73      0.77      6635

Confusion Matrix:
[[3886 1788]
 [  29  932]]
```

```python
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

models = {
    "Decision Tree": dt_predictions,
    "Naive Bayes": nb_predictions
}

for model_name, predictions in models.items():
    print("\n" + "=" * 55)
    print(model_name)
    print("=" * 55)

    print(f"Accuracy: {accuracy_score(y_test, predictions):.4f}")

    print("\nClassification Report:")
    print(classification_report(
        y_test,
        predictions,
        labels=[0, 1],
        target_names=["Retained (0)", "Dropped Off (1)"],
        digits=4,
        zero_division=0
    ))

    print("Confusion Matrix:")
    print(confusion_matrix(
        y_test,
        predictions,
        labels=[0, 1]
    ))
```

```
...
-------------------------------------------------------
Decision Tree
-------------------------------------------------------
Accuracy: 0.9580

Classification Report:
                precision    recall  f1-score   support

   Retained (0)     0.9952    0.9554    0.9749      5674
Dropped Off (1)     0.7870    0.9729    0.8702       961

       accuracy                         0.9580      6635
      macro avg     0.8911    0.9642    0.9225      6635
   weighted avg     0.9651    0.9580    0.9597      6635

Confusion Matrix:
[[5421  253]
 [  26  935]]

-------------------------------------------------------
Naive Bayes
-------------------------------------------------------
Accuracy: 0.7261

Classification Report:
                precision    recall  f1-score   support

   Retained (0)     0.9926    0.6849    0.8105      5674
Dropped Off (1)     0.3426    0.9698    0.5064       961

       accuracy                         0.7261      6635
      macro avg     0.6676    0.8274    0.6584      6635
   weighted avg     0.8985    0.7261    0.7665      6635

Confusion Matrix:
[[3886 1788]
 [  29  932]]
```

Figure 8: Classification Reports and Model Evaluation

Inference: The Decision Tree achieved an accuracy of 95.80%, while Naïve Bayes achieved 72.61%. For the dropped-off class, the Decision Tree obtained precision of 78.70%, recall of 97.29% and an F1-score of 87.02%. Naïve Bayes obtained precision of 34.26%, recall of 96.98% and an F1-score of 50.64%. These results indicate that the Decision Tree provides a better overall balance between identifying dropped-off viewers and avoiding false drop-off predictions. The results remain preliminary until potential data leakage has been fully assessed.

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

ConfusionMatrixDisplay.from_predictions(
    y_test,
    dt_predictions,
    labels=[0, 1],
    display_labels=["Retained", "Dropped Off"],
    cmap="Blues",
    ax=axes[0],
    colorbar=False
)
axes[0].set_title("Decision Tree Confusion Matrix")

ConfusionMatrixDisplay.from_predictions(
    y_test,
    nb_predictions,
    labels=[0, 1],
    display_labels=["Retained", "Dropped Off"],
    cmap="Blues",
    ax=axes[1],
    colorbar=False
)
axes[1].set_title("Naive Bayes Confusion Matrix")

plt.tight_layout()
plt.show()
```

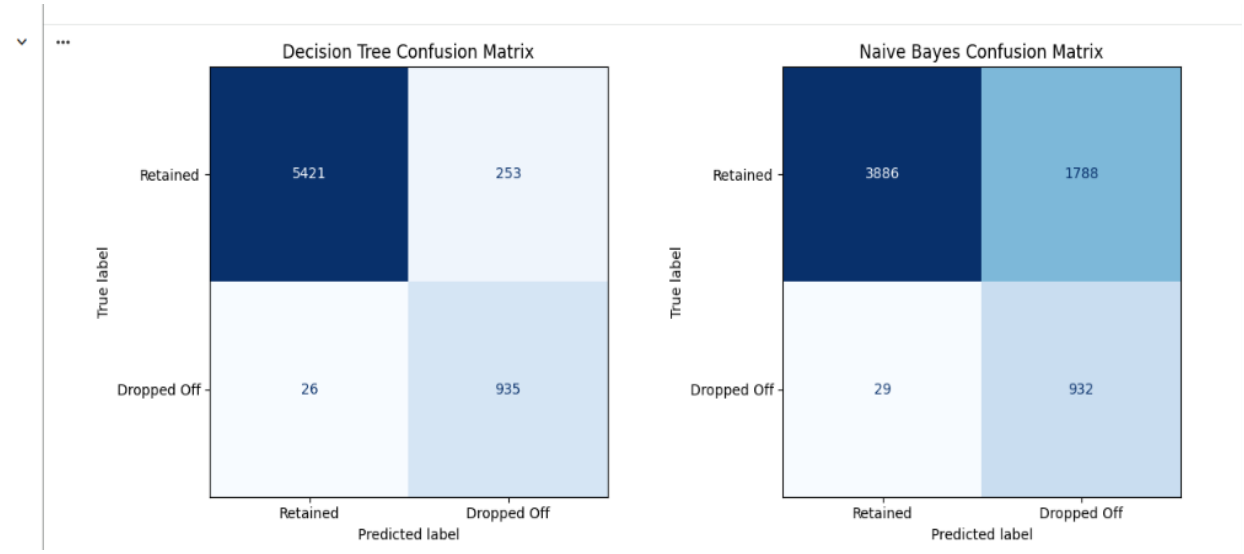


Figure 9: Confusion Matrices of the Two Classification Models

Inference: The Decision Tree correctly classified 5,421 retained records and 935 dropped-off records. It incorrectly classified 253 retained records as dropped-off and missed 26 actual dropped-off records. Naïve Bayes correctly classified 3,886 retained records and 932 dropped-off records, but incorrectly classified 1,788 retained records as dropped-off and missed 29 dropped off records. Therefore, both models identified most actual drop-off records, but the Decision Tree produced substantially fewer false drop-off predictions.

```
# Step 8: Evaluate Decision Tree and Naive Bayes

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Make predictions on unseen testing data
dt_predictions = decision_tree.predict(X_test)
nb_predictions = naive_bayes.predict(X_test)

# Evaluate Decision Tree
print("========== DECISION TREE ==========")
print("Accuracy:", round(accuracy_score(y_test, dt_predictions), 4))
print("\nClassification Report:")
print(classification_report(
    y_test, dt_predictions,
    labels=[0, 1],
    target_names=["Retained (0)", "Dropped Off (1)"],
    zero_division=0
))
print("Confusion Matrix:")
print(confusion_matrix(y_test, dt_predictions, labels=[0, 1]))

# Evaluate Naive Bayes
print("\n========== NAIVE BAYES ==========")
print("Accuracy:", round(accuracy_score(y_test, nb_predictions), 4))
print("\nClassification Report:")
print(classification_report(
    y_test, nb_predictions,
    labels=[0, 1],
    target_names=["Retained (0)", "Dropped Off (1)"],
    zero_division=0
))
print("Confusion Matrix:")
print(confusion_matrix(y_test, nb_predictions, labels=[0, 1]))
```

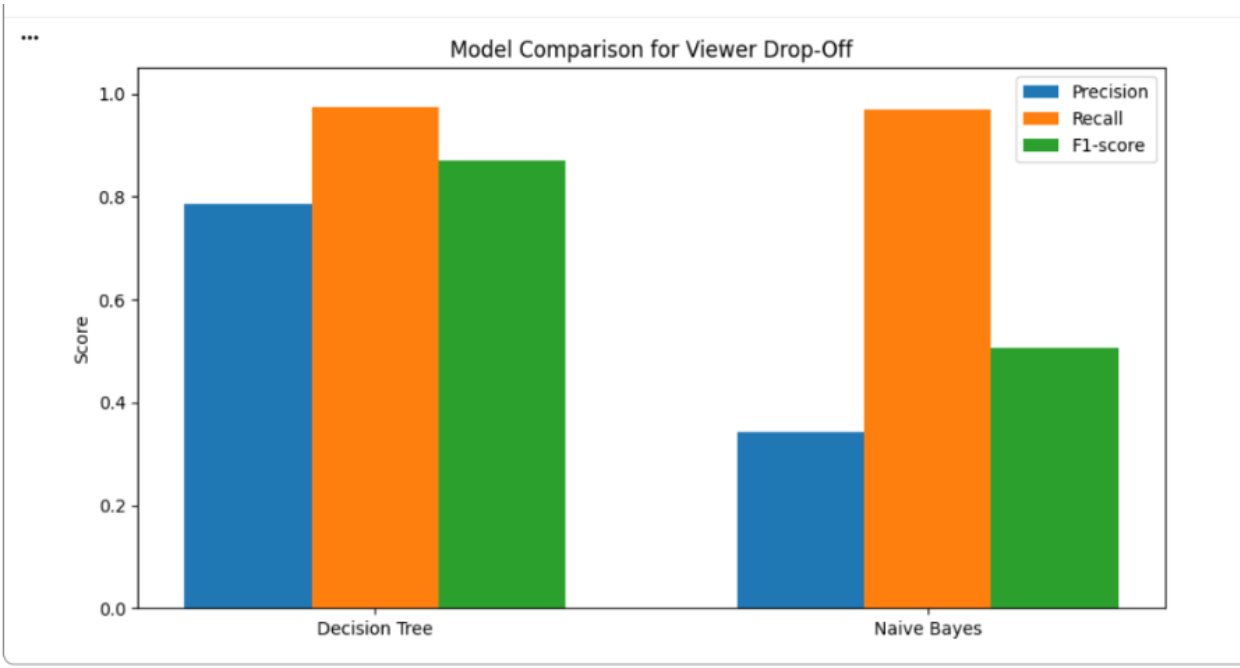


Figure 10: Comparison of Decision Tree and Naïve Bayes Models

Inference: The comparison chart shows that the Decision Tree achieved a higher precision and F1-score for the dropped-off class than Naïve Bayes, while both models achieved similarly high recall. This indicates that the Decision Tree offers a better balance between detecting potential drop-off and reducing incorrect warnings. Based on the current evaluation, the Decision Tree is the preferred model for further testing.

8. How Data Mining Supports the Business Goal?

The results can support business decisions by highlighting patterns associated with viewer drop-off and helping analysts prioritise further investigation. For example, a streaming service could:
review content categories or episode characteristics associated with higher drop-off;
test changes to episode introductions, pacing or content presentation using controlled experiments;
use validated predictions to decide which content or user-experience areas need further analysis;
monitor precision and recall over time and reassess the model when viewing behaviour changes.
The model does not prove cause and effect. Recommendations should be tested with reliable data and controlled evaluation. Any deployment should consider privacy, fairness and the consequences of incorrect predictions.

9. Conclusion:

This case study applied a data mining workflow to an OTT viewing-behaviour dataset to analyse viewer drop-off. After inspecting and preprocessing the data, Decision Tree and Naïve Bayes classifiers were trained and evaluated. Decision Tree performed better overall, achieving 95.80% accuracy and an F1-score of 87.02% for the dropped-off class, compared with 72.61% accuracy and 50.64% F1-score for Naïve Bayes. Decision Tree is the preferred candidate for further validation. However, results remain preliminary until the dataset source, target leakage and timing of input features are verified. The analysis demonstrates how data mining can turn viewing records into insights that may support content review and viewer-retention decisions.